

CSCE 421: Machine Learning

Homework #5

April 17, 2025

Sutton Elliott

UIN:531008819

Email: suelliott77@tamu.edu

This question refers to the textbook “Deep Learning : Foundations and Concepts”.
Consider the autoregressive language model given by (12.31) and repeated below as:

$$p(x_1, \dots, x_N) = \prod_{n=1}^N p(x_n | x_1, \dots, x_{n-1})$$

and suppose that the terms $p(x_n | x_1, \dots, x_{n-1})$ on the right-hand side are represented by general probability tables. Show that the number of entries in these tables grows exponentially with the value of n .

Given the model in the question. Assuming that each variable x_i can take V possible values which represents the number of words in a given vocabulary.

The size of the table for $p(x_n | x_1, \dots, x_{n-1})$:

- To define this probability table, we need to specify the probability of x_n given every possible combination of $x_1, \dots, x_{n-1}$.
- The number of combinations of $(x_1, \dots, x_{n-1})$ is V^{n-1} .
- For each such combination, x_n can take V values.

So, the total number of entries in the full table for $p(x_n | x_1, \dots, x_{n-1})$ is:

$$\# \text{ of entries} = V^{n-1} \times V = V^n$$

Growth with respect to n :

- Since the number of entries is V^n , this shows exponential growth with n , assuming V is fixed and greater than 1.

This question is about transformer language models. You are given a decoder transformer language model in which the conditional distributions $p(x_n | x_1, \dots, x_{n-1})$ are modeled using a transformer. Now you are asked to modify the model to become a tri-gram model in $p(x_n | x_{n-1}, x_{n-2})$ is modeled. Explain how you will modify the model. You may use an example to illustrate your answer.

Transformers typically model full-context dependencies, i.e., each token attends to all previous tokens in the sequence. To convert a decoder transformer into a tri-gram model, we need each token to depend only on the two preceding tokens.

To Modify the Decoder Transformer:

Change the Attention Mask

In a standard decoder transformer, the causal mask ensures that each position n can attend to all previous positions $1, 2, \dots, n-1$.

To create a tri-gram model, we modify the attention mask so that each token at position n only attends to positions $n-2$ and $n-1$. All other positions are masked.

Example:

Given the sequence:

$x = [\text{The, man, went, to, work}]$

Standard Transformer:

- To predict "work" (position 5), the model can attend to "The", "man", "went", and "to".

Tri-gram Transformer:

- To predict "work", the model should only attend to the previous two tokens: "went" and "to".

Position:	1	2	3	4	5
Token:	The	man	went	to	work
Visible to "work":	X	X	✓	✓	X

Thus, we adjust the attention mask so that each token can only "see" the two tokens before it.

Conclusion:

By modifying only the attention mask, we can turn a decoder transformer into a tri-gram model, limiting its context to the two immediately preceding tokens. No changes to the architecture itself are required.

Explain why encoder language models are NOT able to generate sequences.

There are multiple reasons why encoders can't generate sequences:

Bidirectional Context:

- Encoder models look at both past and future tokens during training
- This makes them great at understanding context, but they can't be used for left-to-right generation, because during generation the future tokens are not available yet
- Encoders are used to fill in sequences using the full context

No Autoregressive Mechanism:

- Sequence generation requires predicting token x_n given $x_1, x_2 \dots, x_{n-1}$.
- Encoder models aren't trained to do this. They are built to generate tokens step-by-step

Input-Output Symmetry:

- In encoders, input and output are the same sequence
- For generation, you need a setup where the input grows token by token, which is what decoder models or encoder-decoder models handle.

In short: Encoders are trained for understanding, not for generation.

Coding Task:

GPT for text generation tasks: In this assignment, you will implement a decoder-only Transformer model on the SCAN dataset using PyTorch. The goal of SCAN is to translate commands presented in simplified natural language into a sequence of actions. In this translation task, the generation model will take a command sentence as input and output the corresponding action sequence. For more details, please refer to the dataset available at <https://huggingface.co/datasets/scan>. You can also refer to its original paper at <https://arxiv.org/pdf/1711.00350.pdf> for the introduction of the task. The starting code is provided in the "code" folder. All the architecture and training code is provided, but you need to implement the CSABlock (causal self-attention block) class in the "model.py" file and the generate sample function in the "generate.py" file. Similarly, in this assignment, you must use a GPU.

Requirements are the same as our project including Python, NumPy, PyTorch, plus tqdm and datasets (from Hugging Face). Other packages for transformer implementations are not allowed.

Please submit running report (briefly explain how you complete those functions, capture screen shots of your training/validation loss, test acc etc., anything required in the original assignment) for all coding tasks. And paste training and testing console record as an appendix in your report. You mustn't submit the pre-trained model and the dataset which could be potentially large. Code should be written in a clean and organized way. Comments in the code are required. Questions that only include code without a report and explanations will not be graded. Please make sure all required files are included in your submission by downloading your submission on Canvas and double check it. We will not accept any late submission or any re-submission.

Run the starting code directly to download the SCAN dataset automatically. Read the code, understand the data processing, and answer the following questions: What is a tokenizer? How does a tokenizer process the raw data? What is the size of the vocabulary?

A tokenizer is tool used in natural language processing that splits raw text into smaller units called tokens. Tokens can be words, sub words, or characters, depending on the type of tokenizer. The goal of tokenization is to convert human-readable text into a format that machine learning models can process. The tokenizer will usually perform two tasks: splitting the raw text into individual tokens and after breaking the text into tokens, a tokenizer often maps each token into a unique identifier, which is then used as input to machine learning models.

The tokenizer processes raw text in several stages:

- Vocabulary Creation: First, the tokenizer reads through the raw data and builds a vocabulary of unique tokens (e.g., words, sub words). It tracks the frequency of each token, and each unique token is assigned a unique ID.
- Text Tokenization: The tokenizer then processes the raw text by splitting it into individual tokens. For example, a sentence like "Move the box" might be tokenized into the tokens ["Move", "the", "box"].
- Encoding: Once the text is split into tokens, each token is mapped to its corresponding ID from the vocabulary.
- Decoding: During inference, the tokenizer can also reverse the process to convert token IDs back into human-readable text by mapping each ID back to its token using the reverse vocabulary.

What is the size of the vocabulary? The vocabulary size = 23.

What is the maximum length of the input sequence? How should we determine the maximum length of the input sequence?

The default value for the maximum length or `max_len` is 128.

The maximum length of the input sequence should be long enough to cover the longest sequence you'll encounter during training and generation.

- For input commands: look at the max number of tokens after tokenization (not just word split).
- For output commands: do the same since these are often longer than inputs

A good practice:

- Look at the dataset
- Find max command and action lengths (after tokenizing)
- Set `max_len` slightly above that to be safe — e.g., 128 is safe for SCAN.

Here is a sample code snippet that could be used to find the true max length of any command:

```
from datasets import load_dataset

dataset = load_dataset("scan", "simple")

max_input_len = max(len(example['commands'].split()) for example in dataset['train'])

print("Max input length in tokens:", max_input_len)
```

Implement the CSABlock class in the "model.py" file. Which steps are involved in the self-attention mechanism? Which step is critical to make it "causal" in your code? Why do we need a mask in the forward function of the class "GPT"? Report your training process and results.

CSABlock class implemented.

The self-attention mechanism consists of these key steps:

Linear Projections:

Input embeddings are projected into three different vectors:

- Queries (Q)
- Keys (K)
- Values (V)

Score Computation:

The attention scores are computed by taking the dot product between the query and key vectors:

$$scores = \frac{QK^T}{\sqrt{d_k}}$$

Masking:

For casual models, we apply a casual mask so that each token only attends to previous or current tokens (not future ones)

SoftMax:

The scores are passed through a softmax to get attention weights

Weight Sum:

Attention weights are used to compute a weighted sum of the value vectors, which becomes the output of the self-attention layer

The critical step is applying the causal mask before softmax.

```
att = att.masked_fill(self.mask[:, :, :L, :L] == 0, float('-inf'))
```

- This ensure that the model cannot attend to future tokens during training or generation, preserving autoregressive behavior
- Without this, the model could “peek” at future tokens

We need to a mask to:

- Ensure causality in the decoder-only transformer (GPT).
- Prevent each position from attending to tokens to the right (i.e., future tokens).
- Ensure that generation during inference is consistent with training.

This is especially important for tasks like sequence generation, where the model must predict the next token based only on the past.

Default Training Setup

- n_layer: 2
- n_head: 2
- n_embd: 16
- max_epochs: 60
- batch_size: 32
- learning_rate: 4e-4
- data_split: simple

Results Expectation:

- Loss: should decrease consistently over 60 epochs
- Accuracy: should improve
- Trained Checkpoint: Save to a .pt file

After the first training session, here are a handful of results:

Epoch 1/60	Train loss: 1.35976	Epoch 25/60	Train loss: 0.46527
Epoch 2/60	Train loss: 0.85072	Epoch 26/60	Train loss: 0.48823
Epoch 3/60	Train loss: 0.54828	Epoch 27/60	Train loss: 0.65764
Epoch 4/60	Train loss: 0.84381	Epoch 28/60	Train loss: 0.56480
Epoch 5/60	Train loss: 0.84255	Epoch 29/60	Train loss: 0.45551
Epoch 6/60	Train loss: 0.64031	Epoch 30/60	Train loss: 0.34988
Epoch 7/60	Train loss: 0.55883	Epoch 51/60	Train loss: 0.41300
Epoch 8/60	Train loss: 0.85898	Epoch 52/60	Train loss: 0.41028
Epoch 9/60	Train loss: 0.51138	Epoch 53/60	Train loss: 0.60864
Epoch 10/60	Train loss: 0.65242	Epoch 54/60	Train loss: 0.37600
Epoch 20/60	Train loss: 0.58936	Epoch 55/60	Train loss: 0.40766
Epoch 21/60	Train loss: 0.54673	Epoch 56/60	Train loss: 0.36507
Epoch 22/60	Train loss: 0.65731	Epoch 57/60	Train loss: 0.46234
Epoch 23/60	Train loss: 0.65658	Epoch 58/60	Train loss: 0.42742
Epoch 24/60	Train loss: 0.41118	Epoch 59/60	Train loss: 0.37529

As we can see from the snippet of data above, the training loss decreased over time with as each epoch was trained.

Here is also an example of what the training data looked like:

```
epoch 1 iter 478: train_loss 1.35976. lr 3.9978e-04: 100% | 471/471 [00:26:00:00, 17.68it/s]
test_loss: %f 1.3401576752932567
epoch_valid_loss: 1.3401576752932567, epoch_train_loss: 2.0118814623279935, epoch: 1
Saving at epoch 1: ./cond_gpt/weights/None_simplesplit_2layer_2head_16embd_32bs.pt
step_train_loss: 1.3405423583984375 train_step: 500, learning_rate: 0.0003997489487867487 | 29/471 [00:01:00:19, 22.88it/s]
epoch 2 iter 478: train_loss 0.85072. lr 3.9902e-04: 100% | 471/471 [00:24:00:00, 18.93it/s]
test_loss: %f 0.852481576631654
epoch_valid_loss: 0.852481576631654, epoch_train_loss: 1.0969777299593462, epoch: 2
Saving at epoch 2: ./cond_gpt/weights/None_simplesplit_2layer_2head_16embd_32bs.pt
step_train_loss: 0.9335724711418152 train_step: 1000, learning_rate: 0.0003988910329278014 | 57/471 [00:03:00:27, 14.91it/s]
epoch 3 iter 478: train_loss 0.54828. lr 3.9773e-04: 100% | 471/471 [00:28:00:00, 16.68it/s]
test_loss: %f 0.6938404913218517
epoch_valid_loss: 0.6938404913218517, epoch_train_loss: 0.8387211457432708, epoch: 3
Saving at epoch 3: ./cond_gpt/weights/None_simplesplit_2layer_2head_16embd_32bs.pt
step_train_loss: 0.788762092590332 train_step: 1500, learning_rate: 0.00039742625820294794 | 86/471 [00:04:00:23, 16.39it/s]
epoch 4 iter 478: train_loss 0.84381. lr 3.9590e-04: 100% | 471/471 [00:27:00:00, 17.05it/s]
test_loss: %f 0.6410527161832126
epoch_valid_loss: 0.6410527161832126, epoch_train_loss: 0.75565039339592, epoch: 4
Saving at epoch 4: ./cond_gpt/weights/None_simplesplit_2layer_2head_16embd_32bs.pt
step_train_loss: 0.7076762914657593 train_step: 2000, learning_rate: 0.00039535908601049877 | 116/471 [00:07:00:18, 19.52it/s]
epoch 5 iter 478: train_loss 0.84255. lr 3.9354e-04: 100% | 471/471 [00:28:00:00, 16.58it/s]
test_loss: %f 0.6071824729442596
epoch_valid_loss: 0.6071824729442596, epoch_train_loss: 0.7112186225293153, epoch: 5
Saving at epoch 5: ./cond_gpt/weights/None_simplesplit_2layer_2head_16embd_32bs.pt
```

Implement the generate sample function in the "generate.py" file. What is the generation process? Please explain the process using a concrete example in the dataset.

Generate sample function implemented.

The generation process in a decoder-only Transformer is autoregressive decoding. This means that model generates the output sequence one token at a time, using the previously generated tokens as context at each step. It starts with a known input and then predicts the next token until it reaches a special end-of-sequence token or a maximum length.

Example:

Input Instruction: jump left twice

Target Output: LTURN JUMP LTURN JUMP

Generation process step-by-step:

- Tokenization: The instruction "jump left twice" is converted into token IDs by the tokenizer.
- Model Input: These token IDs are fed into the model as the input context.
- Start generation loop:
 - o The model predicts the first output token: LTURN
 - o This token is appended to the output sequence.
 - o The model then uses the input, and this output token to predict the next token: JUMP
 - o This process repeats:
 - Predict: LTURN
 - Predict: JUMP
 - o After generating the full sequence, the model predicts the <EOS> token, which stops the process.
- Detokenization: The generated token IDs are converted back into the final string: LTURN JUMP LTURN JUMP

The generation process is an iterative prediction loop where the model creates an output token-by-token, using both the input and its own past predictions to guide what comes next. This allows it to produce complex output sequences like those required in the SCAN task.

Re-train your GPT model using different number of layers, number of heads, and number of embeddings. Report your validation loss, time per epoch, and test results in a table. What is the impact of these hyperparameters on the model performance?

When retraining the GPT, here are the new parameters:

- Number of Layers: 4
- Number of Heads: 4
- Number of Embeddings: 24
- Number of Epoch: 10

	Validation Loss	Time Per	Epoch Loss (Results)
Epoch 1	0.91048	1:07	1.69262
Epoch 2	0.61480	1:14	0.78093
Epoch 3	0.51920	1:07	0.64119
Epoch 4	0.44611	1:39	0.57974
Epoch 5	0.41381	1:21	0.53629
Epoch 6	0.36630	1:30	0.50944
Epoch 7	0.35574	1:34	0.48361
Epoch 8	0.334750	1:33	0.47117
Epoch 9	0.32973	1:32	0.46200
Epoch 10	0.32467	1:17	0.45772

Analysis of Hyperparameter Impact:

The retrained model, with increased depth and complexity (4 layers, 4 heads, and 24 embedding dimensions), showed slower training and worse generalization:

- Validation Loss: The best validation loss for this configuration was 0.32467, compared to ~0.26 in the original model. This suggests that the new model may be overfitting or not learning as effectively with the current setup.
- Training Time: Each epoch now takes ~70–90 seconds, compared to 15–30 seconds previously. The increase in layers and heads contributes significantly to the computational cost.
- Performance: Despite the higher model capacity, the performance did not improve, indicating that the model may not benefit from increased complexity at this embedding size or that more regularization/data is needed.

Conclusion:

Changing hyperparameters like the number of layers, attention heads, and embedding dimensions can significantly affect model training time and performance. In this case, increasing these values did not yield better results and instead led to a higher validation loss and slower training, suggesting that the original, smaller configuration was better suited for the task.

Here is a picture of the results as well:

```
epoch 1 iter 470: train loss 0.96589. lr 3.9204e-04: 100% | 471/471 [01:07:00:00, 7.00it/s]
test loss: %f 0.9104856522578113
epoch_valid_loss: 0.9104856522578113, epoch_train_loss: 1.692286933022194, epoch: 1
Saving at epoch 1: ./cond_gpt/weights/None_simplesplit_4layer_4head_24embd_32bs.pt
step_train_loss: 0.9739055837498474 train_step: 500, learning_rate: 0.00039087735563809236 | 29/471 [00:05:01:37, 4.54it/s]
epoch 2 iter 470: train loss 0.66455. lr 3.6530e-04: 100% | 471/471 [01:14:00:00, 6.30it/s]
test loss: %f 0.6148003356636695
epoch_valid_loss: 0.6148003356636695, epoch_train_loss: 0.780930335228104, epoch: 2
Saving at epoch 2: ./cond_gpt/weights/None_simplesplit_4layer_4head_24embd_32bs.pt
step_train_loss: 0.7176550830708313 train_step: 1000, learning_rate: 0.0003607222023680751 | 58/471 [00:07:00:40, 10.12it/s]
epoch 3 iter 470: train loss 0.64247. lr 3.2230e-04: 100% | 471/471 [01:07:00:00, 6.94it/s]
test loss: %f 0.5192016633051746
epoch_valid_loss: 0.5192016633051746, epoch_train_loss: 0.6411922834734501, epoch: 3
Saving at epoch 3: ./cond_gpt/weights/None_simplesplit_4layer_4head_24embd_32bs.pt
step_train_loss: 0.633866548538208 train_step: 1500, learning_rate: 0.000312776048357125 | 87/471 [00:17:01:12, 5.31it/s]
epoch 4 iter 470: train loss 0.57808. lr 2.6720e-04: 100% | 471/471 [01:30:00:00, 4.72it/s]
test loss: %f 0.4461154707197873
epoch_valid_loss: 0.4461154707197873, epoch_train_loss: 0.579749545663785, epoch: 4
Saving at epoch 4: ./cond_gpt/weights/None_simplesplit_4layer_4head_24embd_32bs.pt
step_train_loss: 0.5282747149467468 train_step: 2000, learning_rate: 0.00025234625032986065 | 116/471 [00:23:01:06, 5.35it/s]
epoch 5 iter 470: train loss 0.51683. lr 2.0565e-04: 100% | 471/471 [01:21:00:00, 5.78it/s]
test loss: %f 0.4138124736974824
epoch_valid_loss: 0.4138124736974824, epoch_train_loss: 0.5362901094106606, epoch: 5
Saving at epoch 5: ./cond_gpt/weights/None_simplesplit_4layer_4head_24embd_32bs.pt
step_train_loss: 0.5092141628265381 train_step: 2500, learning_rate: 0.00018612203084319637 | 145/471 [00:19:00:55, 5.89it/s]
epoch 6 iter 470: train loss 0.38951. lr 1.4346e-04: 100% | 471/471 [01:30:00:00, 5.18it/s]
test loss: %f 0.3663087841474785
epoch_valid_loss: 0.3663087841474785, epoch_train_loss: 0.5094494931637101, epoch: 6
Saving at epoch 6: ./cond_gpt/weights/None_simplesplit_4layer_4head_24embd_32bs.pt
step_train_loss: 0.4827767014503479 train_step: 3000, learning_rate: 0.00012143402076577033 | 174/471 [00:37:00:52, 5.67it/s]
epoch 7 iter 470: train loss 0.44591. lr 8.6833e-05: 100% | 471/471 [01:34:00:00, 4.97it/s]
test loss: %f 0.3557433086746144
epoch_valid_loss: 0.3557433086746144, epoch_train_loss: 0.4836092099642298, epoch: 7
Saving at epoch 7: ./cond_gpt/weights/None_simplesplit_4layer_4head_24embd_32bs.pt
step_train_loss: 0.46051740646362305 train_step: 3500, learning_rate: 6.544280163678588e-05 | 203/471 [00:40:01:01, 4.39it/s]
epoch 8 iter 470: train loss 0.52567. lr 4.1329e-05: 100% | 471/471 [01:33:00:00, 5.03it/s]
test loss: %f 0.3347502592599617
epoch_valid_loss: 0.3347502592599617, epoch_train_loss: 0.4711724560859085, epoch: 8
Saving at epoch 8: ./cond_gpt/weights/None_simplesplit_4layer_4head_24embd_32bs.pt
step_train_loss: 0.5344797372817993 train_step: 4000, learning_rate: 4e-05 | 232/471 [00:42:00:28, 8.25it/s]
epoch 9 iter 470: train loss 0.41674. lr 4.0000e-05: 100% | 471/471 [01:32:00:00, 5.10it/s]
test loss: %f 0.3297352281943807
epoch_valid_loss: 0.3297352281943807, epoch_train_loss: 0.4620007077852085, epoch: 9
Saving at epoch 9: ./cond_gpt/weights/None_simplesplit_4layer_4head_24embd_32bs.pt
step_train_loss: 0.4763675928115845 train_step: 4500, learning_rate: 4e-05 | 261/471 [00:43:00:37, 5.63it/s]
epoch 10 iter 470: train loss 0.38105. lr 4.0000e-05: 100% | 471/471 [01:17:00:00, 6.09it/s]
test loss: %f 0.32467287553931104
epoch_valid_loss: 0.32467287553931104, epoch_train_loss: 0.4577287045760266, epoch: 10
Saving at epoch 10: ./cond_gpt/weights/None_simplesplit_4layer_4head_24embd_32bs.pt
PS C:\Users\jortol\OneDrive\Desktop\CSCE 471\H51\code>
```

There are other splits (instead of the “simple” one) of the SCAN dataset

<https://huggingface.co/datasets/scan>. You can use other splits by simply setting the CLI argument “data split” to the names of the splits. Please try to use another split (your choice) and compare your result with the results reported in the original paper

The split that I used for this part of the assignment is the “length” split. The parameters used for this training session will be the same as the retraining in the last question:

- Number of Layers: 4
- Number of Heads: 4
- Number of Embeddings: 24
- Number of Epoch: 10

The only difference being instead of the default “simple” split, “length” used instead. This is done using the command line, python main.py --task train --data_split length.

Here is a comparison of the data for each of the two data splits:

	Simple Data Split			Length Data Split		
	Validation Loss	Time Per	Epoch Loss (Results)	Validation Loss	Time Per	Epoch Loss (Results)
Epoch 1	0.91048	1:07	1.69262	1.01920	1:09	1.77390
Epoch 2	0.61480	1:14	0.78093	0.69574	2:11	0.88232
Epoch 3	0.51920	1:07	0.64119	0.60094	2:17	0.73995
Epoch 4	0.44611	1:39	0.57974	0.53391	1:48	0.66820
Epoch 5	0.41381	1:21	0.53629	0.46618	1:35	0.61363
Epoch 6	0.36630	1:30	0.50944	0.41776	2:11	0.57028
Epoch 7	0.35574	1:34	0.48361	0.39356	2:16	0.54094
Epoch 8	0.334750	1:33	0.47117	0.37297	2:18	0.51953
Epoch 9	0.32973	1:32	0.46200	0.36064	2:19	0.50512
Epoch 10	0.32467	1:17	0.45772	0.35319	2:18	0.49808

Key Observations:

- Validation Loss: While both models showed consistent improvement across epochs, the final validation loss for the "simple" split was 0.32467, slightly better than the 0.35319 on the "length" split. This confirms that the "length" split is more challenging due to its emphasis on compositional generalization — requiring the model to handle longer or more complex sequences than seen during training.
- Time Per Epoch: Training on the "length" split consistently took longer per epoch, with times averaging around 2:15 per epoch, compared to around 1:20 per epoch on the "simple" split. This could be due to increased sequence complexity or internal padding impacting training efficiency.
- Training Loss: The "length" split also resulted in higher overall training loss values compared to "simple". Despite a similar trend in convergence, the model struggled more to reach the same level of confidence in its predictions.

Conclusion

The comparison highlights how dataset structure significantly impacts model performance. The "simple" split allows for easier generalization, leading to faster convergence and lower losses. Meanwhile, the "length" split reveals limitations in the model's ability to generalize to sequences outside the distribution seen during training — a key motivation in the original SCAN paper. This experiment reinforces the importance of evaluating models on more challenging and compositional tasks to assess their true generalization capabilities.

Here is a picture of the results using the length data split:

```
epoch 1 iter 477: train loss 1.89063. lr 3.9204e-04: 100% | 478/478 [01:09:00:00, 6.87it/s]
test loss: %f 1.0192061883431893
epoch valid loss: 1.0192061883431893, epoch train loss: 1.7739080668458548, epoch: 1
Saving at epoch 1: ./cmd_gpt/weights/None_lengthsplit_4layer_4head_24embd_32bs.pt
step_train_loss: 1.89421127761731 train_step: 500, learning_rate: 0.0003911609688571456 | 22/478 [00:04:02:04, 3.65it/s]
epoch 2 iter 477: train loss 0.88864. lr 3.6530e-04: 100% | 478/478 [02:11:00:00, 3.64it/s]
test loss: %f 0.6957475498878567
epoch valid loss: 0.6957475498878567, epoch train loss: 0.8823297347982558, epoch: 2
Saving at epoch 2: ./cmd_gpt/weights/None_lengthsplit_4layer_4head_24embd_32bs.pt
step_train_loss: 0.8219522833824158 train_step: 1000, learning_rate: 0.0003618849697826454 | 44/478 [00:12:02:01, 3.57it/s]
epoch 3 iter 477: train loss 0.72066. lr 3.2230e-04: 100% | 478/478 [02:17:00:00, 3.48it/s]
test loss: %f 0.6009464556420291
epoch valid loss: 0.6009464556420291, epoch train loss: 0.7399512055528713, epoch: 3
Saving at epoch 3: ./cmd_gpt/weights/None_lengthsplit_4layer_4head_24embd_32bs.pt
step_train_loss: 0.5990146994590759 train_step: 1500, learning_rate: 0.00031519931048756974 | 66/478 [00:19:01:54, 3.59it/s]
epoch 4 iter 477: train loss 0.59644. lr 2.6728e-04: 100% | 478/478 [01:48:00:00, 4.39it/s]
test loss: %f 0.5339141024483575
epoch valid loss: 0.5339141024483575, epoch train loss: 0.6682078829619675, epoch: 4
Saving at epoch 4: ./cmd_gpt/weights/None_lengthsplit_4layer_4head_24embd_32bs.pt
step_train_loss: 0.67555489698761 train_step: 2000, learning_rate: 0.0002561394889916723 | 88/478 [00:18:01:16, 5.11it/s]
epoch 5 iter 477: train loss 0.58373. lr 2.0565e-04: 100% | 478/478 [01:35:00:00, 5.02it/s]
test loss: %f 0.4661878667495899
epoch valid loss: 0.4661878667495899, epoch train loss: 0.6136391945963627, epoch: 5
Saving at epoch 5: ./cmd_gpt/weights/None_lengthsplit_4layer_4head_24embd_32bs.pt
step_train_loss: 0.5521933436393738 train_step: 2500, learning_rate: 0.00019102782396736867 | 110/478 [00:25:01:44, 3.52it/s]
epoch 6 iter 477: train loss 0.52944. lr 1.4346e-04: 100% | 478/478 [02:11:00:00, 3.64it/s]
test loss: %f 0.41776577686821975
epoch valid loss: 0.41776577686821975, epoch train loss: 0.5702855893964966, epoch: 6
Saving at epoch 6: ./cmd_gpt/weights/None_lengthsplit_4layer_4head_24embd_32bs.pt
step_train_loss: 0.55876147470398 train_step: 3000, learning_rate: 0.00012688999424838945 | 132/478 [00:36:01:33, 3.70it/s]
epoch 7 iter 477: train loss 0.44170. lr 8.6833e-05: 100% | 478/478 [02:16:00:00, 3.51it/s]
test loss: %f 0.39356755051347947
epoch valid loss: 0.39356755051347947, epoch train loss: 0.54020494097262746, epoch: 7
Saving at epoch 7: ./cmd_gpt/weights/None_lengthsplit_4layer_4head_24embd_32bs.pt
step_train_loss: 0.5669611692428589 train_step: 3500, learning_rate: 7.061048548677742e-05 | 154/478 [00:43:01:29, 3.61it/s]
epoch 8 iter 477: train loss 0.59648. lr 4.1329e-05: 100% | 478/478 [02:18:00:00, 3.46it/s]
test loss: %f 0.37297418382432723
epoch valid loss: 0.37297418382432723, epoch train loss: 0.5195344710823881, epoch: 8
Saving at epoch 8: ./cmd_gpt/weights/None_lengthsplit_4layer_4head_24embd_32bs.pt
step_train_loss: 0.48929288517074585 train_step: 4000, learning_rate: 4e-05 | 176/478 [00:50:01:20, 3.74it/s]
epoch 9 iter 477: train loss 0.57052. lr 4.0000e-05: 100% | 478/478 [02:19:00:00, 3.43it/s]
test loss: %f 0.30864377040774737
epoch valid loss: 0.30864377040774737, epoch train loss: 0.5051206391476189, epoch: 9
Saving at epoch 9: ./cmd_gpt/weights/None_lengthsplit_4layer_4head_24embd_32bs.pt
step_train_loss: 0.30569140424265137 train_step: 4500, learning_rate: 4e-05 | 198/478 [00:56:01:16, 3.66it/s]
epoch 10 iter 477: train loss 0.40842. lr 4.0000e-05: 100% | 478/478 [02:18:00:00, 3.45it/s]
test loss: %f 0.353191988892464
epoch valid loss: 0.353191988892464, epoch train loss: 0.42008481578796976, epoch: 10
Saving at epoch 10: ./cmd_gpt/weights/None_lengthsplit_4layer_4head_24embd_32bs.pt
PS C:\Users\sutto\OneDrive\Desktop\CSCE_421\VA6\code>]]
```

<https://arxiv.org/pdf/1711.00350.pdf>. What is the split you choose? What is the type of evaluation that the split is designed for? What insights do you get from the comparison?

Chosen Split: For this experiment, the "length" split of the SCAN dataset was selected.

Purpose of the "Length" Split: The "length" split is designed to evaluate a model's ability to generalize to sequences longer than those seen during training. Specifically, it tests compositional generalization by requiring the model to interpret and execute commands that are more complex or longer than the training examples.

Insights from the Comparison:

- Validation Loss: The model trained on the "length" split exhibited a higher validation loss compared to the "simple" split. This indicates that the model found it more challenging to generalize to longer sequences, highlighting the increased difficulty of the "length" split.
- Training Time: Training on the "length" split took longer per epoch. This is likely due to the increased complexity and length of the sequences, which require more computational resources and time to process.
- Model Performance: The final validation loss for the "length" split was higher than that of the "simple" split, suggesting that the model's performance was better on the simpler, more straightforward sequences.

Conclusion: The comparison underscores the importance of evaluating models on more challenging splits like "length" to assess their true generalization capabilities. While the model performed well on the "simple" split, its performance declined on the "length" split, revealing limitations in handling more

complex, longer sequences. This aligns with the findings reported in the original SCAN paper, emphasizing the need for models that can generalize compositionally beyond their training data.